ChatGPT

Step-by-Step Plan for Modeling Japan's Zonal Electricity Market Bid Stack

Step 1: Construct Generation Stack per Region (Plant Data & Constraints)

Japan's power system is divided into nine regional zones (excluding isolated Okinawa), split between the 50 Hz eastern grid and 60 Hz western grid. Interconnection capacity between these zones is limited by a few HVDC links, so each zone often operates as a semi-independent market [1]. For instance, the map above shows a snapshot where Eastern Japan's zones had a higher day-ahead price (~9.3 JPY/kWh) while Western zones were ~6.9 JPY/kWh, reflecting congestion on the east-west links [2]. Thus, we will construct a separate generation *bid stack* (merit order) for each region, acknowledging the limited inter-zone transmission. Each region's stack will be a sorted list of generation units by **ascending marginal cost**, representing how supply offers aggregate to meet demand [3].

To build a *realistic* stack, we will include key plant-level attributes for every sizable generator in the zone:
- **Plant Type & Fuel:** (e.g. coal, LNG, oil, nuclear, hydro, solar, etc.) which determines its variable cost and behavior.
- **Capacity and Availability:** Maximum output (MW) of each unit and whether it's online or on maintenance in each interval.
- **Marginal Cost Curve:** Derived from fuel cost (fuel price * heat rate) plus variable O&M. For hydro/renewables, this cost is near-zero. We may use a single cost per unit or a piecewise cost curve if partial-load efficiency varies.
- **Ramping Constraints:** Maximum ramp-up/down rate (MW per 30 min) to model that some thermal plants cannot instantly go from 0 to full output. This will be considered when moving from one interval to the next – a unit may not be able to increase output enough to meet a sharp demand spike, so higher-cost fast units (e.g. gas turbines) fill the gap.
- **Operational Constraints:** Minimum stable output (for thermal units that cannot run below a threshold), minimum up/down times (once started or stopped), and startup costs. These constraints mean the static merit order might not strictly be followed at all times – e.g. a coal plant may stay on (must-run at min output) even if its cost is higher than some peakers, due to shutdown costs. We plan to incorporate such constraints in the simulation logic (e.g. keeping units on if shutting down is suboptimal).

Using these attributes, we construct the merit-order stack for each zone: sort all available generation by **bid price** (initially equal to marginal cost for a competitive baseline). This sorted stack essentially represents the supply curve. During each 30-min interval, the intersection of this supply curve with regional demand will determine the clearing price (the marginal unit's offer) [3]. We will prepare the data structures so that for any given demand level we can quickly compute which plants are dispatched and what the marginal cost (price) is. Initially, we will ignore inter-regional transfers (assuming each zone meets demand internally up to interconnector limits); later we can introduce an option to model limited **inter-zone flow** constraints if needed (e.g. allow exchanging surplus power between East and West up to known link capacities). The

outcome of this step is a framework to represent each region's generation stack, setting the stage for simulation. All subsequent steps will utilize these per-zone stacks as the core supply model.

Step 2: Source and Build Regional Generator Metadata

To populate each region's stack, we need a **comprehensive generator metadata dataset**. We will gather plant-level data from publicly available Japanese sources: ideally open data from the EPCOs (regional power companies), OCCTO, METI, or other regulatory bodies. For example, the Tokyo area's utility (TEPCO) publishes detailed information on its power stations, including capacities and fuel types (e.g. TEPCO's Chiba thermal power station totals 4,380 MW across multiple LNG-fired units) [4]. Similar data can be obtained for other regions via:

- **Utility Disclosure:** Annual reports or websites of the major power companies (e.g. KEPCO in Kansai, Chubu Electric, etc.) often list their generation facilities and capacities. These can be scraped or collected (TEPCO's online data lists each plant's capacity, efficiency, and fuel [5] [6], and other EPCOs provide plant lists as well).
- **OCCTO / METI Data:** The government and OCCTO aggregate supply capacity information. METI's statistics (e.g. *"Number and Output of Electricity Utility Power Plants"*) provide the count and capacity of plants by type each year [7] [8]. As of 2022, Japan had ~269 GW of installed capacity across 1,000+ generation entities [8] – our model will prioritize the largest units that make up the bulk of capacity in each region (since tiny generators have negligible effect on wholesale prices). OCCTO's annual reports and supply plans also list planned new units and retirements [9] [10], which can enrich our dataset.
- **Renewable and Specialized Databases:** For certain fuel types, dedicated datasets exist. For instance, **Japan Beyond Coal** provides an online map and data for every coal-fired unit (with location, size, status) [11]. Similar sources may exist for solar farms or wind (though solar is highly distributed; we might use aggregate capacities). The **Renewable Energy Institute** aggregates data on installed renewable capacity and generation by region, which helps attribute how much solar/wind is in each zone [12].
- **Cross-verification:** We will cross-verify capacities with multiple sources. The Federation of Electric Power Companies (FEPC) statistics or JEPIC reports can serve as references for total regional capacities by fuel type, ensuring our sum of plant data matches official figures. This avoids omissions of any major plant.

Using these sources, we will compile a **generator table** with columns such as: *Plant ID/Name, Region (market zone), Owner, Fuel Type, Capacity (MW), Heat Rate or Efficiency, Variable Cost (¥/MWh at full output), Ramp Rate (MW/interval), Min Up/Down Time, etc.* Each plant is assigned to one of the regional zones based on its location or the utility that operates it. We will also flag special units: e.g. **Interconnector facilities** (frequency converter stations between East/West) – these aren't generation but affect transfers; **Pumped hydro** units (which act as both load and generation) – we'll include them with their generation capacity and treat pumping load separately in demand data. Nuclear units will be included with zero/low marginal cost but note their availability (many may be offline or output-limited due to regulations, which we reflect via an availability schedule).

This metadata set will be the foundation of the model. We will ensure it's comprehensive and structured (possibly in a CSV or database) so that the simulation code can easily query all plants in a given region and

retrieve their parameters. By the end of this step, we will have a **complete generation fleet dataset for each region**, enabling realistic merit-order construction and later calibration.

Step 3: Integrate 30-Minute Demand and Generation Time-Series

With the supply stack in place, we next gather **time-series data** for each region, at 30-minute intervals, to drive the simulation. This includes: (a) the regional electricity *demand* (load) and (b) time-varying output from certain generation categories (notably renewables and any must-run units). We will leverage newly available data from OCCTO and the regional TSOs for this step. Since 2016, OCCTO has published detailed supply-demand performance data for the 10 areas (9 main regions plus Okinawa) on a monthly basis [13] [14]. These data sets typically include: total demand, and a breakdown of generation by source (e.g. thermal, hydro, solar, wind) for each region in each time interval, or at least weekly graphs of such data. We will obtain the **half-hourly demand** for each zone (either directly from OCCTO's data download or via the individual utility data portals – for example, TEPCO Power Grid provides a real-time demand graph and data archive for the Tokyo area).

In addition to demand, the model benefits from knowing **must-take generation** in each interval: for example, solar PV output and wind output are largely non-dispatchable and are supplied first. OCCTO's data includes weekly records of solar and wind generation and curtailment by area [15] [16]. We will integrate this by treating solar/wind as predetermined supply: i.e. at each timestep, subtract the actual solar/wind generation (if known) from the demand to get the net demand that thermal and other controllable plants must meet. This ensures our simulation respects actual renewable output and its effect on prices (e.g. midday solar output reduces demand for thermal plants, often lowering price). Similarly, if available, we will incorporate hydro generation schedules – though hydro (especially pumped storage) can be optimized, we might use actual hydro output as given to avoid needing a separate optimization for reservoir management. Nuclear generation will be treated as a fixed output per plant when running (since nuclear plants typically operate at full power when available). For each nuclear unit, we can use reported capacity factors or monthly output to allocate a generation profile (or use OCCTO's data if it lists nuclear output per area).

All this will be done at **30-minute resolution** to match the market's timeframe (JEPX clears the day-ahead market in 30-min slots [3], and balancing mechanisms also operate sub-hourly). We will gather data for a substantial period (e.g. a year or multiple years of history) to have enough variation for calibration. This means handling ~17,520 intervals per year per region. Using structured data (e.g. a Pandas DataFrame), we will create a time-indexed table for each region with columns like: *Demand, Solar_gen, Wind_gen, Nuclear_gen, etc.*.

Quality checks and processing: We will ensure demand and generation data are aligned in time (Japan does not observe daylight savings, so 48 intervals per day consistently). If any data is missing or only available as images/PDFs, we may use OCR or manual digitization – however, given OCCTO's initiative, numerical CSV data should be obtainable [14]. We'll also incorporate **inter-regional flow** data if available: OCCTO provides 5-minute interconnection flow data [17], which could be averaged to 30-min. This allows us to know when a region was importing or exporting. In an initial model, we might treat net imports as an addition to local supply (reducing effective demand on local plants) up to the known capacity of interconnectors. Alternatively, we first model each region in isolation and check if simulated prices diverge significantly in connected regions; if so, we'll introduce constraints to share power until the interconnector limits (which could be a later enhancement).

By integrating these time-series data, we set up the **simulation inputs**: for each time step and region, we know how much power needs to be produced by the dispatchable stack, and we have a baseline of must-run or must-take generation. This is crucial for realistic simulation and calibration: it ensures the model sees the same demand peaks and renewable troughs that the real market faced.

Step 4: Calibrate the Merit-Order Model to Historical Prices

With the static merit-order (from Step 1/2) and the demand timeline (Step 3), we can run a **basic dispatch simulation** and calibrate it against historical market prices. First, we will simulate the *Day-Ahead market clearing* under the assumption of perfect competition (all generators bid at marginal cost). For each 30-minute interval, our code will: take the region's net demand (after renewables), sort that region's available plants by their marginal cost, then "dispatch" plants in order until the demand is met. The marginal cost of the last unit dispatched becomes the *simulated price* for that interval (analogous to JEPX clearing price) [3]. We will obtain a time series of simulated prices for each region.

Next, we compare this simulated price curve to the **historical price curve** from the day-ahead market. (JEPX publishes area prices for each 30-min slot – we have access to this data, e.g. via JEPX or a compiled dataset [18].) If our model were perfect, the simulated and actual price curves would match, but initially there will be discrepancies. We will systematically **calibrate the model parameters** to reduce these gaps:

- **Fuel Cost and Heat Rate Adjustments:** Even though we input fuel prices (which the user has) and unit efficiencies, there may be nuances (e.g. older plants might effectively have higher marginal costs than expected due to maintenance or fuel transport costs). If simulated prices are consistently lower than actual, it could indicate we need to raise marginal costs for certain generators. We might introduce cost multipliers for certain categories (e.g. coal units might effectively bid higher due to emissions costs or risk aversion). These multipliers will be tuned so that the base-load price level matches historical (for example, if nighttime prices in our simulation are ¥5/kWh but historically ¥6/kWh, we may slightly increase coal/gas costs to raise the floor price). Similarly, if simulated peak prices are too low, perhaps peaker units' costs are underestimated – we'll verify fuel assumptions (LNG spot price spikes, etc.) and adjust the cost curve shape (e.g. gas turbines might have a higher heat rate at full load, effectively higher marginal cost).

- **Plant Availability and Outages:** The historical price at times may spike because a large unit was offline (e.g. an unplanned outage or maintenance). If our model dispatches that unit (thus keeping price low) but in reality it was unavailable (leading to a higher-cost unit setting price), we need to incorporate that information. We will use the generation data to infer outages: e.g., if a 1000 MW plant shows zero output during a high-demand period in actual data, we mark it as **unavailable** in the model for that period. By doing so, the model will instead use the next-costly units, raising the simulated price closer to actual. We can automate this by comparing each plant's maximum output in a period to its capacity; if it never output anything when it normally would be economic, assume it was down. This calibration of availability ensures the supply stack in each interval reflects reality (including maintenance schedules, deratings, etc.).

- **Ramping and Startup Effects:** We will adjust how we handle ramp constraints if needed. For example, if a steep morning demand ramp in reality caused prices to jump (perhaps because some slow coal plants couldn't come online fast enough and expensive gas peakers set the price), but our simulation simply turned on coal (because marginal cost ordering would), then we need to enforce

ramp limits. We might simulate the dispatch chronologically, carrying over the last interval's committed units and limiting how quickly they can change output. If price spikes still aren't high enough compared to actual, it could be that in reality some units did not startup in time (perhaps due to startup lead times). We can introduce a rule that certain plants require notice to turn on (e.g. a cold coal plant can't respond within a half-hour window). Calibrating this might involve ensuring the timing of price spikes aligns: e.g., if actual prices spike in the early evening when solar drops, our model should show a similar spike if no additional units can ramp up fast enough, forcing high-cost units or scarcity pricing to occur.

- **Hydro/Pumped Storage Dispatch:** We will also calibrate the use of storage and hydro. These resources can greatly affect price by shifting supply. If our model is naive, it might dispatch hydro whenever it's cheapest, but in reality operators use it strategically at peak price times. We can use the actual generation profiles of pumped storage to guide our model: for instance, if actual data shows pumped hydro output during the top 5% price hours, we ensure our simulation also holds hydro for those times. We might set up a simple optimization or heuristic: allocate limited hydro energy to the highest demand (or highest price error) periods, which should flatten extreme peaks and bring simulation closer to actual price shape.

This calibration step will likely be iterative: adjust parameters, rerun the simulation, and see if the **error** (difference between simulated and actual prices) improves. We will examine metrics like mean absolute error, and also qualitative shape features: Does the model capture the highest price hours? Does it ever produce a price spike when the real market did not (indicating oversensitivity)? Through this process, we refine the merit-order model to closely reflect historical market behavior under an assumption of marginal-cost bidding. At this point, if done well, the **merit-order model alone may explain a large portion of price variation** – any remaining systematic deviations could be due to strategic bidding, which we address next.

Step 5: Extend the Model for Strategic Bidding Behavior

The next step is to incorporate **strategic bidding** into the simulation to capture behavior beyond pure marginal-cost offers. In Japan's market, although competition has increased, the dominant generators in each region (often the former incumbent utility or J-Power) could have the ability and incentive to raise prices by offering some capacity at prices above cost, especially during tight demand periods. We will implement an extension of the model to simulate such strategic behavior in two possible ways:

- **Mark-Up Method:** We introduce a markup on the bids of certain generators or on certain fractions of their capacity. For example, we might program the large thermal plants of the dominant firm to bid a portion of their output at a higher price. One heuristic is to tie markup to the reserve margin: when expected reserve margin in a time slot is low (demand is close to total available supply), generators know the price will likely be set by the highest-cost unit, so they can bid above cost without being undercut. Concretely, we can set a rule such as: if demand > 90% of capacity, then all remaining online units bid at a price cap (or significantly above marginal cost) for any output beyond their first 50% of capacity. This effectively withholds some supply, creating a steep rise in the supply curve near maximum output, which can produce price spikes resembling those in real markets. We will calibrate the level of markups by region – e.g. in regions where one company controls most capacity, higher markups might be observed.

- **Game-Theoretic Approach (optional advanced):** For a more principled strategy, we can model a simplified **oligopoly game**. For instance, treat the largest few firms in each region as players that choose bid prices to maximize profit given the residual demand. We could use a Supply Function Equilibrium or Cournot model approximation. In practice, implementing a full game theory solution is complex, so we might approximate it by iterative best-response: simulate the market, identify if any generator could have profitably raised its bid (without being displaced), adjust its bid up, and repeat until no one can profit more. This would result in an equilibrium set of bids that is higher than marginal costs when the market is tight. However, given the scope, a simpler parametric approach (like the markup method above) may suffice for now, with parameters tuned to mimic historical outcomes.

We will incorporate these strategic features as **optional modules**: the code can toggle between *Perfect Competition mode* (all bids at MC) and *Strategic mode* (with markups). In strategic mode, instead of using each plant's marginal cost as the bid price, we will use an adjusted bid price. For example, we create a new field in the generator data called `bid_price` which initially equals marginal cost, and then apply rules: e.g., for each interval, if a generator's owner has X% market share of the capacity currently online, increase its `bid_price` by a certain factor for that interval. We might also introduce *fixed* strategic bid blocks: e.g., some plants might bid part of their capacity at very high prices (essentially withholding it unless the price is extreme). This can simulate how companies hold back some reserve to drive prices up then only sell at high prices.

We will use historical price-volume data to guide this: JEPX publishes cleared volumes and the bid curves. If accessible, the bid curve data [19] [18] could tell us how steep the supply curve gets near the top – a very steep curve near the peak load is a sign of strategic withholding. We can calibrate our artificial markups so that the simulated supply curve yields a similar price elasticity (e.g., a small increase in demand leads to a large price jump when near full capacity).

Once implemented, we will test the strategic model on known events. For instance, if there were instances of price spikes or divergence between day-ahead and estimated marginal costs, the strategic model should be able to produce higher prices in those intervals than the pure merit-order model did. We'll verify that adding strategic bidding improves the fit to actual prices (without severely overshooting in other periods). This extension will allow our simulation to not only use costs, but also capture **market power effects** that are present in the real Japanese power market.

Step 6: Calibrate and Validate Simulated Prices vs. Historical Price Curves

Finally, we will perform a comprehensive calibration of the **full model (with strategic behavior)** to the historical price time series, and then validate its performance. We have two sets of historical price data to consider: the primary one is the **JEPX day-ahead prices** for each region (area prices), and secondarily, if available, the **balancing (real-time) market prices** or imbalance prices. The goal is for our simulated price curves to statistically match the historical ones in each region, in terms of distributions, daily patterns, and responses to events.

Our calibration process will likely involve fine-tuning:
- **Strategic Bidding Parameters:** We will adjust the markup magnitudes or thresholds defined in Step 5 so

that the frequency and magnitude of price spikes in the simulation align with reality. For example, if historically the Tokyo area price frequently hit ~200 ¥/kWh in winter 2021 (when supply was very tight), our model should produce similar spikes under those conditions. We'll tune how much capacity is withheld or bid up to get that result. If our initial strategic assumptions overshoot (producing higher prices than actual), we dial back the markups.
- **Error Analysis:** We will compute error metrics between simulated and actual price for each region (e.g. mean error, standard deviation ratio, and the distribution of residuals). We will particularly focus on **peak hours and low-demand hours**: Does the model overshoot during low-demand times (could indicate too high bid for base units) or undershoot peaks (too much supply being offered cheaply)? We address these by targeted adjustments – e.g., if off-peak prices are too high, maybe we've overestimated fuel cost or included a must-run cost that wasn't actually there; if peak prices are still too low, perhaps not enough strategic withholding is modeled or some peaking units that in reality set a very high price (like oil-fired units) are not accurately represented and need a higher bid price in the model.

- **Incorporating Balancing Market Data:** If we have data for balancing market (real-time) prices, we will use it to calibrate the model's representation of scarcity and outages. The balancing market in Japan started around 2021 and can have extreme prices when supply-demand unexpectedly diverge. Day-ahead model might not capture that unless we introduce random outages or forecast errors. We can simulate some **stochastic scenarios**: randomly remove a generator or mis-predict renewables to see how high prices can go, and ensure the range overlaps with balancing prices. However, since the question focus is zonal day-ahead, this is a secondary check. Primary is to match the day-ahead *average* price behavior.

To validate the calibration, we will overlay simulated prices on historical supply-demand charts. The figure above shows an example week of actual data: the stacked generation by source (colored areas) meets demand, and the JEPX price (black line) spikes when demand peaks or renewable output dips. We expect our model, when given the same demand and renewables inputs, to produce a price curve that closely follows the actual black line [3]. We will create such plots for each region and visually inspect that the timing and magnitude of price changes align. Additionally, we'll check statistical properties: e.g., the mean price, the 95th percentile price, and duration of price > X in simulation vs reality. Once the model reproduces these well, we consider it calibrated.

Finally, we will **test the model's predictive capability** on data it hasn't been explicitly fit to (if possible). For example, use year 2024 data to calibrate, then simulate 2025 (using actual demand/weather and fuel prices for 2025) and see if it predicts the price reasonably – this would validate that our calibration is not overfit. We'll also examine each region: some regions might be easier to fit (e.g. large ones with many plants), while small ones (like isolated Hokkaido) might have unique issues (like local network constraints or fewer plants leading to more volatility). We will document any systematic deviation we cannot capture (for instance, if there are administrative price caps or out-of-market actions that keep prices down, our model might overshoot those unless we explicitly include such mechanisms). Those can be noted as limitations or future improvements.

At the end of this step, we should have a **fully calibrated simulation model** that, given 30-min demand and renewables data and fuel costs, can output a price trajectory per region that matches the historical day-ahead market prices with high fidelity. This sets the stage for using the model for future scenarios or analysis (e.g. what-if simulations, testing strategic behavior under different conditions, etc.).

Proposed Modular Codebase Structure

To implement the above plan, we will create a modular and scalable codebase. The project will be organized into components that reflect the logical steps of the modeling process, enabling easy maintenance and extension. Below is a recommended structure and key modules:

- **Data Ingestion Module** (`data_ingestion.py`): Responsible for reading all input data and converting it into structured formats. This will include functions to load generator metadata (e.g. from CSV or Excel compiled from OCCTO/EPCO sources) and functions to load time-series data (regional demand, renewable generation, historical prices). We will use pandas DataFrames for tabular data. For example, a DataFrame for generators with columns like `region, plant_id, type, fuel, capacity, heat_rate, ramp_rate, min_up_time, ...`, and a DataFrame for demand with columns `timestamp, region, demand_MW, solar_MW, wind_MW, net_import_MW, ...`. This module will also handle any required data cleaning (converting units, filling missing values) and possibly caching of data for performance.

- **Generator Model and Merit Order** (`generator_stack.py` or an OOP class `GenerationFleet`): This part constructs the merit-order stack for each region from the generator metadata. If using an object-oriented approach, we might define a `PowerPlant` class (with attributes like capacity, marginal_cost, etc., and methods to determine if available, increase output within ramp limits, etc.) and a `Region` class containing a list of `PowerPlant` and functions to sort them by bid price. In functional approach, we'd have a function like `build_merit_order(region, time, mode='competitive')` that returns a sorted list of plants (or a supply curve). This module will implement the logic for setting **bid prices**: e.g., one function to compute marginal cost from fuel price and heat rate, and another to adjust that to a bid price (incorporating markups or strategic behavior flags). It will use parameters for strategic behavior that can be configured (e.g., a dictionary of markups per plant or rules based on reserve margin). This separation makes it easy to switch between pure cost-based bids and strategic bids by simply toggling a parameter.

- **Dispatch & Simulation Engine** (`dispatch_simulation.py`): This contains the core algorithm to simulate the market clearing for each interval. One implementation might be a loop or vectorized operation that for each time step calls something like `merit_order = build_merit_order(region, t, mode)`, then accumulates generation until demand is met, and identifies the marginal plant and price. We might implement an optimization-based approach for better fidelity: for example, use a linear programming solver (PuLP or Pyomo) for unit commitment – minimize total cost subject to meeting demand and unit constraints (this would yield dispatch and shadow price = marginal cost). However, a full UC model for all intervals might be heavy; instead, we can implement a greedy or myopic dispatch with some heuristics for ramping (or rolling 48-interval optimization for each day to decide commitment, which strikes a balance between realism and complexity). The engine will respect **ramping** by linking consecutive intervals: e.g., keep track of each unit's last output and ensure we don't dispatch beyond what ramp allows; if needed, incorporate logic for startup decisions (start a unit if demand cannot be met by already running units within ramp limits + quick-start peakers). The output of the simulation engine will be both the *dispatched generation per plant* and the *market clearing price* for each interval and region. This can be stored as time-indexed results, e.g. a DataFrame of simulated prices with columns for each region, and perhaps a 3D structure or separate DataFrame for generation by plant if needed for analysis.

- **Calibration & Optimization Tools** (`calibration.py`): A module containing functions to calibrate model parameters against actual data. For instance, a function `tune_fuel_costs(simulated_prices, actual_prices, generator_data)` that might adjust fuel cost multipliers or heat rates to minimize the error. We might employ algorithms like gradient descent or simple iterative search since the parameter space (fuel cost multipliers, markup factors, outage adjustments) is manageable. This could also utilize an optimizer: for example, set up an optimization problem to minimize sum of squared errors between simulated and actual prices by adjusting a set of parameters (with constraints to keep them within realistic bounds). We can leverage libraries like SciPy's optimizers for this continuous tuning, or even heuristic approaches (like a genetic algorithm if the objective space is rough). Additionally, this module will have functions to inject known outages: e.g., `apply_outage_schedule(generator_data, actual_gen_timeseries)` to mark units offline when needed. The calibration module will likely be used in an interactive/iterative way rather than fully automated at first – e.g., we run simulation, observe errors, then use these tools to adjust and re-run. Over time, these can be automated into a loop for fine-tuning.

- **Validation & Analysis** (`analysis.py` or Jupyter notebooks): While not strictly part of the core simulation code, we will have scripts or notebooks to visualize and compare results. These will plot simulated vs actual price curves, load-duration curves, supply stack utilization, etc. This is crucial for verifying model accuracy. We might also include statistical tests or calculations (mean error, correlation) in this analysis module. By separating it, we keep the simulation code clean but ensure we have a way to continually verify outputs.

- **Configuration & Parameters** (`config.py` or JSON/YAML config files): The model will have numerous parameters (e.g. fuel prices, strategic markup levels, etc.). We will maintain these in a config file or module, making it easy to change without altering code. For instance, fuel price time series could be loaded here or paths to data files defined. If the user has a lot of fuel price data, we might integrate that as an input time series for fuel cost (so the model can reflect, say, daily LNG price changes). The config can also hold things like interconnector capacities between zones, which might be needed if we simulate flows.

- **Modularity and Scalability:** The codebase will be designed so that adding a new region or updating data is straightforward. For example, if new plant data comes in, one can update the CSV and re-run without code changes. If we want to model a future year, we just supply new demand and fuel forecasts to the input. The separation of concerns (data vs simulation vs calibration) ensures that improvements in one area (say, better data for renewables) can be integrated without rewriting the dispatch logic. We will document each module with clear input/output interfaces. Moreover, we can make use of parallel computing if needed (since each region's simulation is largely independent aside from any coupling via interconnectors). For instance, we could run simulations for all regions in parallel threads or processes to speed up calibration, given the large number of intervals. This approach will allow the codebase to handle increasing complexity (like more plants, longer time spans, or finer time resolution) with minimal refactoring.

In summary, this modular structure aligns with the plan's steps: we have dedicated parts for data, for building the stack, for running the simulation (with both competitive and strategic modes), and for calibration/validation. The use of structured data (as in the provided example dataset image) ensures our simulation can ingest and produce organized outputs that are easy to analyze. By following this plan and

code structure, we will create a robust model of Japan's zonal power market that can be calibrated to historical performance and used for future scenario analysis, all while being maintainable and extensible.

Sources: Japan Electric Power Exchange data portal [18] ; OCCTO energy data releases [13] [14] ; TEPCO power station data [4] ; JEPIC report (2023) [8] ; Renewable Energy Institute statistics [12] ; CME Group market overview [3] [2] ; Wikipedia (Japan grid) [1] .

---

[1] Electricity sector in Japan - Wikipedia
https://en.wikipedia.org/wiki/Electricity_sector_in_Japan

[2] [3] Introduction to the Japanese power market - CME Group
https://www.cmegroup.com/education/articles-and-reports/introduction-to-the-japanese-power-market.html

[4] [5] [6] Electricity Supply Facilities | TEPCO Illustrated | TEPCO Illustrated
https://www.tepco.co.jp/en/corpinfo/illustrated/electricity-supply/thermal-e.html

[7] [9] [10] epijJepic2025_copy.pdf
file://file-CVfX9jnpmnzaXGCD6FYyUM

[8] jepic.or.jp
https://www.jepic.or.jp/en/data/pdf/epijJepic2023.pdf

[11] MAP & DATA | Japan Beyond Coal 石炭火力発電所を2030年までに ...
https://beyond-coal.jp/en/map-and-data/

[12] [17] Japan's Power Supply/Demand and Power Flow of Interregional Interconnections: Data Formated into Visualized Charts and Map | Statistics | Renewable Energy Institute
https://www.renewable-ei.org/en/activities/statistics/20200619.php

[13] [14] [15] [16] OCCTO has started to publish electricity demand data and graphs | ISEP Energy Chart
https://isep-energychart.com/en/1084/

[18] [19] Day Ahead Market | Trading Market Data | Trading Information | JEPX
https://www.jepx.jp/en/electricpower/market-data/spot/